

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Курсовая работа
По дисциплине
“Информационные системы”

Этап 2

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель:
Мартин Райла

Санкт-Петербург, 2025г

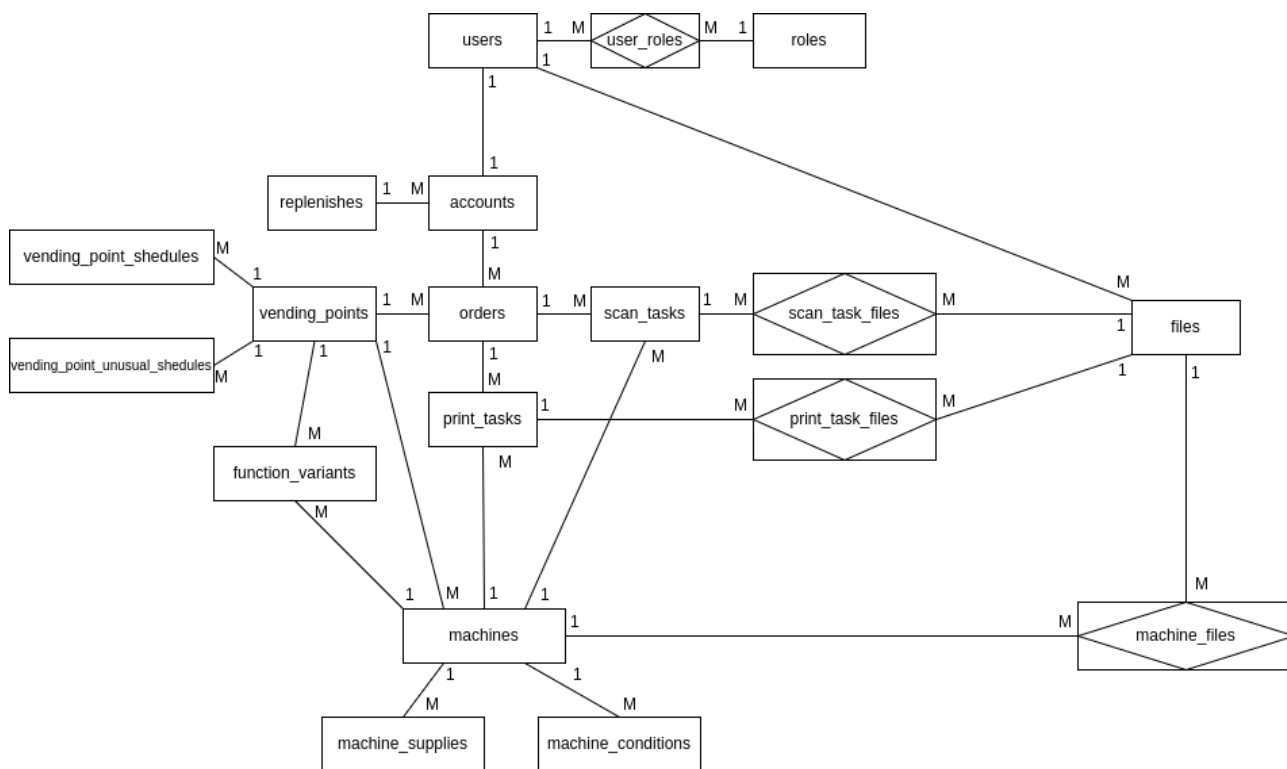
Задание

Этап 2:

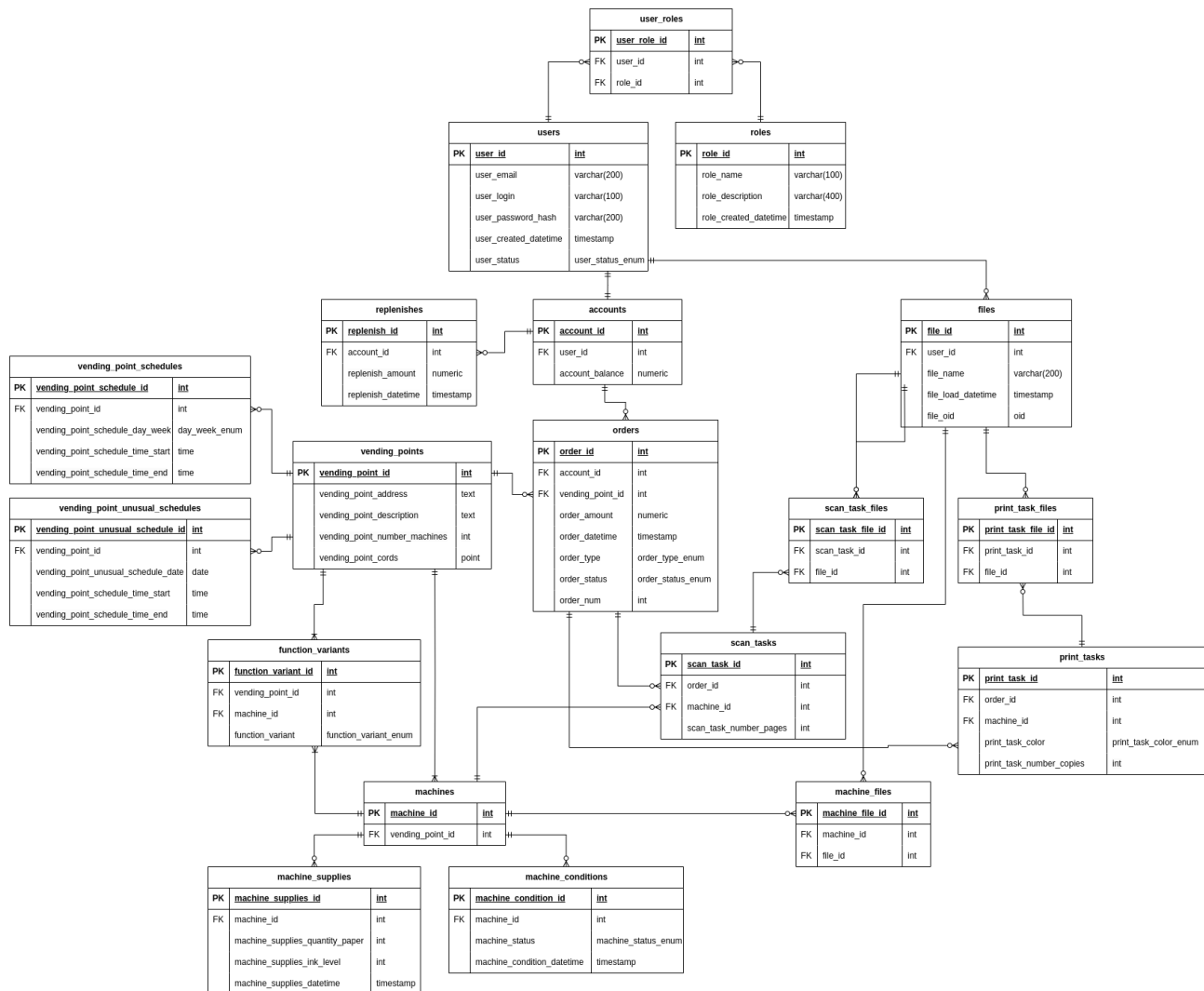
1. Сформировать ER-модель базы данных (на основе описаний предметной области и прецедентов из предыдущего этапа).
2. ER-модель должна:
 1. а. включать в себя не менее 10 сущностей;
 2. б. содержать хотя бы одно отношение вида «многие-ко-многим».
3. Согласовать ER-модель с преподавателем. На основе ER-модели построить даталогическую модель.
4. Реализовать даталогическую модель в реляционной СУБД PostgreSQL.
5. Обеспечить целостность данных при помощи средств языка DDL и триггеров.
6. Реализовать скрипты для создания, удаления базы данных, заполнения базы тестовыми данными.
7. Предложить pl/pgsql-функции и процедуры, для выполнения критически важных запросов (которые потребуются при последующей реализации прецедентов).
8. Создать индексы на основе анализа использования базы данных в контексте описанных на первом этапе прецедентов. Обосновать полезность созданных индексов для реализации представленных на первом этапе бизнес-процессов.
9. Составить отчет.

Ход работы

ER-диаграмма предметной области



Даталогическая модель



Скрипты для создания, удаления базы данных, заполнения базы тестовыми данными

Весь исходный код скриптов, можно найти в репозитории — <https://github.com/stoneshik/is-coursework-2/tree/main>

Скрипт создания базы данных (create.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/create.sql>

Скрипт очистки базы данных (delete.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/delete.sql>

Скрипт заполнения базы тестовыми данными в малом объеме (insert.sql):
<https://github.com/stoneshik/is-coursework-2/blob/main/insert.sql>

Скрипт заполнения базы тестовыми данными в большом объеме > 1Гб

(script_generate_data.sql):

https://github.com/stoneshik/is-coursework-2/blob/main/script_generate_data.sql

Pl/pgsql-функции и процедуры

Список функций и процедур:

- `get_role_id_by_name` — вспомогательная функция для создания нового пользователя, получение id роли по ее названию.
- `create_new_user` — функция создания нового пользователя.
- `create_new_user_trigger` — триггер для создания нового пользователя. Сначала пользователю назначается роль user, затем создается новый счет с нулевым балансом и связывается с пользователем.
- `replenish_account` — функция для триггера пополнения счета пользователя.
- `replenish_account_trigger` — триггер для пополнения счета пользователя, значение счета обновляется на сумму предыдущего значения со значением суммы пополнения.

Скрипт создания функций и процедур (triggers.sql):

```
-- триггер для создания нового пользователя
-- 1) Пользователю назначается роль user
-- 2) Создается новый счет с нулевым балансом и связывается с пользователем
CREATE OR REPLACE FUNCTION get_role_id_by_name(varchar(100))
  RETURNS TABLE(role_id integer) AS $$
  SELECT role_id::integer FROM roles WHERE role_name = $1;
$$ LANGUAGE sql;
```

```
CREATE OR REPLACE FUNCTION create_new_user()
  RETURNS trigger AS $$
DECLARE
  user_id_var integer;
  role_id_var integer;
BEGIN
  user_id_var = NEW.user_id;
  role_id_var = get_role_id_by_name('user');
  INSERT INTO user_roles(user_role_id, user_id, role_id) VALUES
    (default, user_id_var, role_id_var);
  INSERT INTO accounts(account_id, user_id, account_balance) VALUES
    (default, user_id_var, 0);
  RETURN NEW;
END
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER create_new_user_trigger
  AFTER INSERT ON users
  FOR EACH ROW
  EXECUTE FUNCTION create_new_user();
```

```
-- триггер для пополнения счета пользователя
-- 1) Значение счета обновляется на сумму предыдущего значения со значением суммы пополнения
CREATE OR REPLACE FUNCTION replenish_account()
  RETURNS trigger AS $$
DECLARE
  account_id_var integer;
  replenish_amount_var numeric;
BEGIN
  account_id_var = NEW.account_id;
  replenish_amount_var = NEW.replenish_amount;
```

```

UPDATE accounts set account_balance = (account_balance + replenish_amount_var) WHERE account_id =
account_id_var;
RETURN NEW;
END
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE TRIGGER replenish_account_trigger
AFTER INSERT ON replenishes
FOR EACH ROW
EXECUTE FUNCTION replenish_account();

```

Ссылка в репозитории - <https://github.com/stoneshik/is-coursework-2/blob/main/triggers.sql>

Создание индексов, обоснование полезности созданных индексов

Список индексов:

- user_email_index_hash — индекс для быстрого поиска email, для авторизации.
- user_login_index_hash — индекс для быстрого поиска по логину для авторизации.
- order_type_index_btree — индекс для быстрого поиска заказов по типу (печать, сканирование).
- order_status_index_btree — индекс для быстрого поиска заказов по их статусу.
- vending_point_unusual_schedule_date_index_hash — индекс для быстрого поиска расписания по дате
- function_variant_index_btree — индекс для быстрого поиска доступных вариантов по типу варианта (черно-белая печать, цветная печать, сканирование)
- machine_supplies_datetime_index_btree — индекс для ускорения получения последних по времени данных о запасах вендингового аппарата
- machine_condition_datetime_index_btree — индекс для ускорения получения последних по времени данных о состоянии вендингового аппарата

Создание нужных индексов (create_indexes.sql):

```

-- Создание индексов
CREATE INDEX user_email_index_hash ON users USING hash(user_email);
CREATE INDEX user_login_index_hash ON users USING hash(user_login);

CREATE INDEX order_type_index_btree ON orders USING btree(order_type);
CREATE INDEX order_status_index_btree ON orders USING btree(order_status);

CREATE INDEX vending_point_unusual_schedule_date_index_hash ON vending_point_unusual_schedules
USING hash(vending_point_unusual_schedule_date);

CREATE INDEX function_variant_index_btree ON function_variants USING btree(function_variant);

CREATE INDEX machine_supplies_datetime_index_btree ON machine_supplies USING
btree(machine_supplies_datetime);
CREATE INDEX machine_condition_datetime_index_btree ON machine_conditions USING
btree(machine_condition_datetime);

```

Ссылка в репозитории - <https://github.com/stoneshik/is-coursework-2/blob/main/triggers.sql>

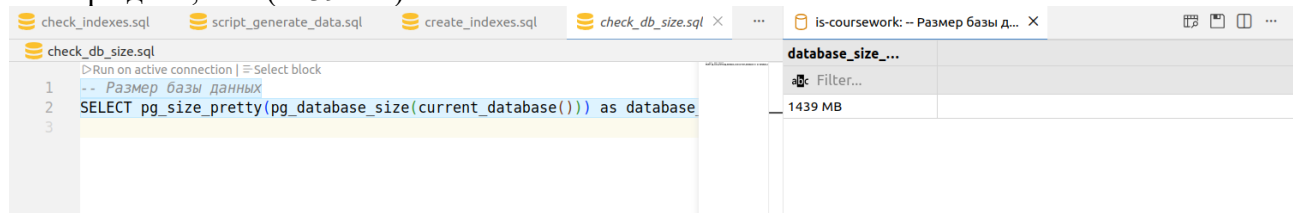
Для доказательства эффективности использования описанных ранее индексов заполним базу данных большим объемом данных (>1 Гб) при помощи скрипта script_generate_data.sql (https://github.com/stoneshik/is-coursework-2/blob/main/script_generate_data.sql).

Запрос проверки размера базы данных:

-- Размер базы данных

```
SELECT pg_size_pretty(pg_database_size(current_database())) as database_size_bytes;
```

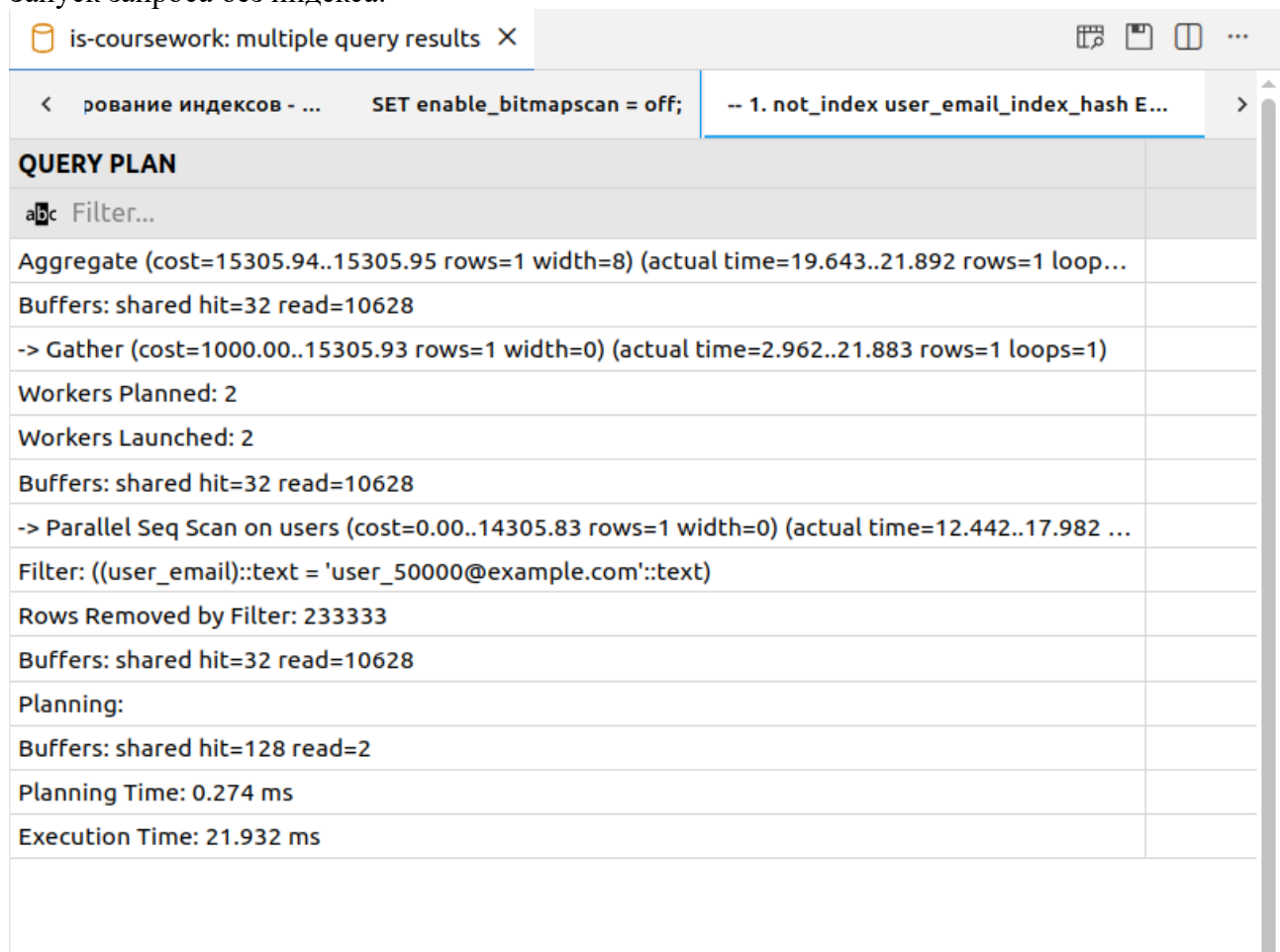
Размер бд – 1,4 Гб (1439 Мб)



Выполним скрипт проверки эффективности индексов, в нем сначала делаются запросы выборки с выключенными индексами и затем точно такие же запросы со включенными индексами. Для анализа выполнения используется Explain Analyze. Также между повторным выполнением запросов очищаем кеш плана выполнения при помощи *DISCARD PLANS*. Скрипт проверки - https://github.com/stoneshik/is-coursework-2/blob/main/check_indexes.sql

1) Проверка user_email_index_hash:

Запуск запроса без индекса:



Запуск запроса с индексом:

is-coursework: multiple query results X

< второй части... SET enable_bitmapscan = on; -- 1. index user_email_index_hash EXPLA... -- 2. in >

QUERY PLAN

Filter...

Aggregate (cost=4.45..4.46 rows=1 width=8) (actual time=0.053..0.053 rows=1 loops=1)

Buffers: shared hit=1 read=3

-> Index Only Scan using users_user_email_key on users (cost=0.42..4.44 rows=1 width=0) (actua...

Index Cond: (user_email = 'user_50000@example.com'::text)

Heap Fetches: 0

Buffers: shared hit=1 read=3

Planning Time: 0.052 ms

Execution Time: 0.069 ms

2) Проверка user_login_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X

< -- 2. not_index user_login_index_hash EX... -- 3. not_index order_type_index_btree E... -- 4. not_i >

QUERY PLAN

Filter...

Aggregate (cost=15305.94..15305.95 rows=1 width=8) (actual time=20.727..22.491 rows=1 loop...

Buffers: shared hit=128 read=10532

-> Gather (cost=1000.00..15305.93 rows=1 width=0) (actual time=3.776..22.484 rows=1 loops=1)

Workers Planned: 2

Workers Launched: 2

Buffers: shared hit=128 read=10532

-> Parallel Seq Scan on users (cost=0.00..14305.83 rows=1 width=0) (actual time=13.494..19.124 ...

Filter: ((user_login)::text = 'user_75000'::text)

Rows Removed by Filter: 233333

Buffers: shared hit=128 read=10532

Planning:

Buffers: shared hit=3

Planning Time: 0.091 ms

Execution Time: 22.509 ms

Запуск запроса с индексом:

is-coursework: multiple query results X	
< iscan = on; -- 1. index user_email_index_hash EXPLA...	-- 2. index user_login_index_hash EXPLAI...
QUERY PLAN	
Filter...	
Aggregate (cost=4.45..4.46 rows=1 width=8) (actual time=0.021..0.022 rows=1 loops=1)	
Buffers: shared hit=1 read=3	
-> Index Only Scan using users_user_login_key on users (cost=0.42..4.44 rows=1 width=0) (actual...	
Index Cond: (user_login = 'user_75000'::text)	
Heap Fetches: 0	
- Buffers: shared hit=1 read=3	
Planning Time: 0.019 ms	
Execution Time: 0.027 ms	

3) Проверка order_type_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X	
< -- 2. not_index user_login_index_hash EX...	-- 3. not_index order_type_index_btree E... -- 4. not_i >
QUERY PLAN	
Filter...	
Finalize Aggregate (cost=76828.45..76828.46 rows=1 width=8) (actual time=174.431..176.429 ro...	
Buffers: shared hit=128 read=47052	
-> Gather (cost=76828.23..76828.44 rows=2 width=8) (actual time=174.360..176.422 rows=3 loo...	
Workers Planned: 2	
Workers Launched: 2	
- Buffers: shared hit=128 read=47052	
-> Partial Aggregate (cost=75828.23..75828.24 rows=1 width=8) (actual time=172.883..172.884 r...	
Buffers: shared hit=128 read=47052	
-> Parallel Seq Scan on orders (cost=0.00..73221.17 rows=1042827 width=0) (actual time=0.013.....	
Filter: (order_type = 'print'::order_type_enum)	
Rows Removed by Filter: 833794	
Buffers: shared hit=128 read=47052	
Planning:	
Buffers: shared hit=56	
Planning Time: 0.130 ms	
Execution Time: 176.454 ms	

Запуск запроса с индексом:

is-coursework: multiple query results X	
< -- 3. index order_type_index_btree EXPL...	-- 4. index order_status_index_btree EXP... -- 5. index >
QUERY PLAN	
Filter...	
Finalize Aggregate (cost=41282.87..41282.88 rows=1 width=8) (actual time=110.327..112.644 ro...	
Buffers: shared hit=6 read=2106	
-> Gather (cost=41282.66..41282.87 rows=2 width=8) (actual time=110.321..112.639 rows=3 loo...	
Workers Planned: 2	
Workers Launched: 2	
- Buffers: shared hit=6 read=2106	
-> Partial Aggregate (cost=40282.66..40282.67 rows=1 width=8) (actual time=108.669..108.670 r...	
Buffers: shared hit=6 read=2106	
-> Parallel Index Only Scan using order_type_index_btree on orders (cost=0.43..37675.59 rows=...	
Index Cond: (order_type = 'print'::order_type_enum)	
Heap Fetches: 0	
Buffers: shared hit=6 read=2106	
Planning Time: 0.029 ms	
Execution Time: 112.670 ms	

4) Проверка order_status_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📊 ...
<	ash EX... -- 3. not_index order_type_index_btree E...	-- 4. not_index order_status_index_btree... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=75939.23..75939.24 rows=1 width=8) (actual time=160.133..161.933 ro...		
Buffers: shared hit=224 read=46956		
-> Gather (cost=75939.02..75939.23 rows=2 width=8) (actual time=160.072..161.927 rows=3 loo...		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=224 read=46956		
-> Partial Aggregate (cost=74939.02..74939.03 rows=1 width=8) (actual time=158.236..158.237 r...		
Buffers: shared hit=224 read=46956		
-> Parallel Seq Scan on orders (cost=0.00..73221.17 rows=687140 width=0) (actual time=0.017..1...		
Filter: (order_status = 'completed'::order_status_enum)		
Rows Removed by Filter: 1111468		
Buffers: shared hit=224 read=46956		
Planning:		
Buffers: shared hit=3		
Planning Time: 0.091 ms		
Execution Time: 161.956 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📊 ...
<	-- 3. index order_type_index_btree EXPL...	-- 4. index order_status_index_btree EXP... -- 5. index... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=27542.41..27542.42 rows=1 width=8) (actual time=74.197..76.236 rows...		
Buffers: shared hit=6 read=1405 written=1070		
-> Gather (cost=27542.19..27542.40 rows=2 width=8) (actual time=74.154..76.230 rows=3 loops=1)		
Workers Planned: 2		
Workers Launched: 2		
Buffers: shared hit=6 read=1405 written=1070		
-> Partial Aggregate (cost=26542.19..26542.20 rows=1 width=8) (actual time=72.367..72.367 ro...		
Buffers: shared hit=6 read=1405 written=1070		
-> Parallel Index Only Scan using order_status_index_btree on orders (cost=0.43..24824.34 rows...		
Index Cond: (order_status = 'completed'::order_status_enum)		
Heap Fetches: 0		
Buffers: shared hit=6 read=1405 written=1070		
Planning Time: 0.066 ms		
Execution Time: 76.254 ms		

5) Проверка vending_point_unusual_schedule_date_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X	
< -- 5. not_index vending_point_unusual_s... -- 6. not_index function_variant_index_b... -- 7. not_i >	
QUERY PLAN	
a1c Filter...	
Aggregate (cost=35.39..35.40 rows=1 width=8) (actual time=0.011..0.012 rows=1 loops=1)	
-> Seq Scan on vending_point_unusual_schedules (cost=0.00..35.38 rows=7 width=0) (actual tim...	
Filter: (vending_point_unusual_schedule_date = (CURRENT_DATE - '10 days'::interval))	
Planning:	
Buffers: shared hit=50 read=1	
- Planning Time: 0.140 ms	
Execution Time: 0.021 ms	

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📑 ...
< ➤ EXPL...	-- 4. index order_status_index_btree EXP...	-- 5. index vending_point_unusual_sched... >
QUERY PLAN		
abc Filter...		
Aggregate (cost=35.39..35.40 rows=1 width=8) (actual time=0.003..0.003 rows=1 loops=1)		
-> Seq Scan on vending_point_unusual_schedules (cost=0.00..35.38 rows=7 width=0) (actual tim...		
Filter: (vending_point_unusual_schedule_date = (CURRENT_DATE - '10 days'::interval))		
Planning:		
Buffers: shared hit=1		
Planning Time: 0.052 ms		
Execution Time: 0.012 ms		

6) Проверка function_variant_index_hash:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 5. not_index vending_point_unusual_s...	-- 6. not_index function_variant_index_b...	-- 7. not_i >
QUERY PLAN		
abc Filter...		
Aggregate (cost=2811.12..2811.14 rows=1 width=8) (actual time=11.561..11.562 rows=1 loops=1)		
Buffers: shared hit=811		
-> Seq Scan on function_variants (cost=0.00..2686.00 rows=50050 width=0) (actual time=0.009.....		
Filter: (function_variant = 'color_print'::function_variant_enum)		
Rows Removed by Filter: 99937		
- Buffers: shared hit=811		
Planning:		
Buffers: shared hit=33		
Planning Time: 0.076 ms		
Execution Time: 11.570 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📑 ...
<	-- 6. index function_variant_index_btree ...	-- 7. index machine_supplies_datetime_i... -- 8. index >
QUERY PLAN		
abc Filter...		
Aggregate (cost=1177.30..1177.31 rows=1 width=8) (actual time=6.655..6.656 rows=1 loops=1)		
Buffers: shared hit=1 read=44 written=44		
-> Index Only Scan using function_variant_index_btree on function_variants (cost=0.29..1052.17 ...)		
Index Cond: (function_variant = 'color_print'::function_variant_enum)		
Heap Fetches: 0		
Buffers: shared hit=1 read=44 written=44		
Planning Time: 0.028 ms		
Execution Time: 6.667 ms		

7) Проверка machine_supplies_datetime_index_btree:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📑 ...
<	isual_s... -- 6. not_index function_variant_index_b...	-- 7. not_index machine_supplies_dateti... >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=40469.78..40469.79 rows=1 width=8) (actual time=185.902..187.769 ro...)		
Buffers: shared hit=64 read=15860		
-> Gather (cost=40469.57..40469.78 rows=2 width=8) (actual time=185.822..187.764 rows=3 loo...)		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=64 read=15860		
-> Partial Aggregate (cost=39469.57..39469.58 rows=1 width=8) (actual time=184.312..184.312 r...)		
Buffers: shared hit=64 read=15860		
-> Parallel Seq Scan on machine_supplies (cost=0.00..39361.50 rows=43228 width=0) (actual tim...)		
Filter: ((machine_supplies_datetime <= now()) AND (machine_supplies_datetime >= (now() - '30 ...		
Rows Removed by Filter: 799142		
Buffers: shared hit=64 read=15860		
Planning:		
Buffers: shared hit=52 read=3		
Planning Time: 0.232 ms		
Execution Time: 187.788 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 6. index function_variant_index_btree ...	-- 7. index machine_supplies_datetime_i...	-- 8. index >
QUERY PLAN		
abc Filter...		
Aggregate (cost=3474.74..3474.75 rows=1 width=8) (actual time=22.719..22.720 rows=1 loops=1)		
Buffers: shared hit=6 read=281 written=281		
-> Index Only Scan using machine_supplies_datetime_index_btree on machine_supplies (cost=0...		
Index Cond: ((machine_supplies_datetime >= (now() - '30 days'::interval)) AND (machine_supplie...		
Heap Fetches: 0		
Buffers: shared hit=6 read=281 written=281		
Planning:		
Buffers: shared hit=4		
Planning Time: 0.081 ms		
Execution Time: 22.733 ms		

8) Проверка machine_condition_datetime_index_btree:

Запуск запроса без индекса:

is-coursework: multiple query results X		🔍 📄 📑 ...
< -- 8. not_index machine_condition_dateti...	DISCARD PLANS;	-- Включаем индексы для е >
QUERY PLAN		
abc Filter...		
Finalize Aggregate (cost=32510.61..32510.62 rows=1 width=8) (actual time=141.992..143.806 ro...		
Buffers: shared hit=64 read=12675		
-> Gather (cost=32510.40..32510.61 rows=2 width=8) (actual time=141.864..143.799 rows=3 loo...		
Workers Planned: 2		
Workers Launched: 2		
- Buffers: shared hit=64 read=12675		
-> Partial Aggregate (cost=31510.40..31510.41 rows=1 width=8) (actual time=140.315..140.316 r...		
Buffers: shared hit=64 read=12675		
-> Parallel Seq Scan on machine_conditions (cost=0.00..31489.00 rows=8559 width=0) (actual ti...		
Filter: ((machine_condition_datetime <= now()) AND (machine_condition_datetime >= (now() - '...		
Rows Removed by Filter: 660283		
Buffers: shared hit=64 read=12675		
Planning:		
Buffers: shared hit=38 read=3		
Planning Time: 0.162 ms		
Execution Time: 143.829 ms		

Запуск запроса с индексом:

is-coursework: multiple query results X	
< _btree ...	-- 7. index machine_supplies_datetime_i...
	-- 8. index machine_condition_datetime_...
QUERY PLAN	
abc Filter...	
Aggregate (cost=690.61..690.62 rows=1 width=8) (actual time=4.051..4.052 rows=1 loops=1)	
Buffers: shared hit=4 read=52 written=52	
-> Index Only Scan using machine_condition_datetime_index_btree on machine_conditions (cost...	
Index Cond: ((machine_condition_datetime >= (now() - '7 days'::interval)) AND (machine_conditi...	
Heap Fetches: 0	
Buffers: shared hit=4 read=52 written=52	
Planning:	
Buffers: shared hit=8	
Planning Time: 0.072 ms	
Execution Time: 4.069 ms	

Таблица сравнения выполнения запросов без индексов и с индексами:

№	С индексами или без индексов	Execution Time (ms)	Разница Execution Time (ms)	Разница Execution Time (%)
1	Без индекса	21,932	21,863	99,69%
	С индексом	0,069		
2	Без индекса	22,509	22,482	99,88%
	С индексом	0,027		
3	Без индекса	176,454	63,784	36,15%
	С индексом	112,670		
4	Без индекса	161,956	85,702	52,92%
	С индексом	76,254		
5	Без индекса	0,021	0,009	42,86%
	С индексом	0,012		
6	Без индекса	11,570	4,903	42,38%
	С индексом	6,667		
7	Без индекса	187,788	165,055	87,89%
	С индексом	22,733		
8	Без индекса	143,829	139,76	97,17%
	С индексом	4,069		

Вывод по использованию индексов:

По планам выполнения запросов видно, то, что индексы используются. При этом все создаваемые индексы значительно оптимизируют время выполнения запросов, поэтому их использование оправдано.

Вывод

В ходе выполнения данного этапа курсовой работы была составлена ER-диаграмма предметной области и также составлена даталогическая модель для дальнейшей реализации в СУБД PostgreSQL. Также была реализована даталогическая модель в реляционной СУБД PostgreSQL. Целостность данных была обеспечена при помощи средств языка DDL, и также были добавлены триггеры для обеспечения комплексной целостности. Также был проведен анализ созданной базы данных и на его основе для оптимизации запросов, были созданы индексы.